

Algorithms

Authors: Shaleen Baral

Contents

1. Matchings	2
1.1. Bipartite Maximum Matching	2
1.2. Hopcroft-Karp Algorithm	4
1.3. Min-Cost Bipartite Perfect Matching, LP Relaxation	6
1.4. <i>Mise en Place</i> : Ski-Rental Problem	8
1.5. Online Matching	8
1.6. Algebraic Algorithms	8

1. Matchings

1.1. Bipartite Maximum Matching

Definition 1.1.1 (matching): A *matching* in an undirected graph is an edge set such that every vertex belongs to either one (*matched vertex*) or $\emptyset$ (*free vertex*) edges of the matching.

Maximum Matching Problem: Given a graph G , find a matching with as many edges as possible.

Given a graph G and a suboptimal matching M , how should we improve it?

Definition 1.1.2: An M -alternating component is an edge set that forms a connected subgraph of G of maximum degree 2, in which every degree-2 vertex belongs to exactly one edge of M .

A connected subgraph of G of maximum degree 2 must be either a cycle or a path. It is a cycle iff every vertex has degree 2. A proof for this can be furnished by considering the longest path in the subgraph.

Definition 1.1.3 (M -augmenting path): Suppose G is a graph and M is a matching. Then, an M -augmenting path in G is a path P in G that starts and ends at free vertices and alternates between edges in M and not in M .

The following are some easy but useful properties of an M -augmenting path.

Lemma 1.1.1: Let P be an M -augmenting path. Then

- P has odd length;
- $|P \setminus M| = |M \cap P| + 1$.

Proof:

- Let $P = v_0 v_1 \dots v_k$. Then $v_i v_{i+1} \in M$ iff $i \equiv 1 \pmod{2}$. As v_k is free, we have $v_{k-1} v_k \notin M$ which implies $(k-1) \equiv 0 \pmod{2}$ so that $\ell(P) = k \equiv 1 \pmod{2}$.
- Let $P = v_0 v_1 \dots v_k$. It suffices to show that $P' = v_0 v_1 \dots v_{k-1}$ is such that $|P' \setminus M| = |P' \cap M|$. For this just consider the $P' \cap M \rightarrow P' \setminus M$ bijection defined by $v_i v_{i+1} \mapsto v_{i-1} v_i$.

□

Proposition 1.1.1: If C is an M -alternating path then $M \oplus C$ is a matching such that

$$|M \oplus C| = |M| + |C \setminus M| - |M \cap C|.$$

Proof: We first show that $M \oplus C$ is a matching. Consider a vertex $v \in M \oplus C$. It suffices to prove that v is adjacent to exactly one edge of $M \oplus C$. Note that $M \oplus C = (M \setminus C) \sqcup (C \setminus M)$ (these induced subgraphs are vertex-disjoint). If $v \in M \setminus C$ then v is adjacent to exactly one edge in $M \setminus C$ as M is a matching. Similarly, if $v \in C \setminus M$ then there are two cases to consider. First, if v is free then it is only adjacent to one edge in C . Second, if v is matched in M then it must be adjacent to exactly one edge in $M \cap C$ and one edge in $C \setminus M$. Consequently, v is adjacent to exactly one edge in $C \setminus M$.

Thus,

$$\begin{aligned} |M \oplus C| &= |(M \setminus C) \sqcup (C \setminus M)| \\ &= |M \setminus C| + |C \setminus M| \\ &= |M| + |C \setminus M| - |M \cap C|. \end{aligned}$$

□

Corollary 1.1.1.1: If P is an M -augmenting path then $M \oplus P$ is a matching and

$$|M \oplus P| = |M| + 1.$$

Furthermore, every vertex matched in M is also matched in $M \oplus P$.

Proof: As P is an M -alternating component, $M \oplus P$ is a matching. Furthermore, as $|P \setminus M| = |M \cap P| + 1$ we have $|M \oplus P| = |M| + 1$.

Furthermore, $V(M) \subseteq V(M \oplus P)$. Let $v \in M$. Note that if $v \notin P$, then clearly $v \in M \setminus P \subseteq M \oplus P$. Otherwise, $v \in P$ and as v is not M -free, it must be an interior vertex that is adjacent to an edge in $P \setminus M$ which implies $v \in P \setminus M \subseteq M \oplus P$.

□

In fact, M -augmenting paths completely characterizes the maximum matching problem.

Lemma 1.1.2: If M and M' are matchings in G then $M \oplus M'$ is a union of M -alternating components.

Proof: Every $v \in M \oplus M'$ has degree $\in \{1, 2\}$ as every vertex has degree at most 1 in each of M, M' (with degree 1 in at least one or the other). Furthermore, a vertex $v \in M \oplus M'$ can only be adjacent to at most one edge in M as M is a matching. Thus, $M \oplus M'$ is a union of M -alternating components.

□

Lemma 1.1.3: If M and M' are matchings in G with $|M| < |M'|$ then $M \oplus M'$ contains an M -augmenting path.

Proof: $M \oplus M'$ is a union of M -alternating components. An M -alternating component where all vertices have degree-2 must be a cycle. Since the edges must alternate between edges in M and in M' , the cycle must have an equal number of edges from M and M' . Since $|M| < |M'|$ we must then have an M -alternating component that is a path and contains more edges from M' than M .

□

This motivates a family of algorithms for the maximum matching problem.

Computing a Maximum-Matching

```

1 initialize  $M = \emptyset$ 
2 repeat
3    $P \leftarrow$  find an  $M$ -augmenting path
4    $M \leftarrow M \oplus P$ 
5 until there is no  $M$ -augmenting path
```

This algorithm terminates in at most $\mathcal{O}(n/2)$ iterations as each iteration increases the size of the matching by 1 and a maximum matching has size at most $n/2$.

Different strategies for finding the M -augmenting path P yield different algorithms.

Definition 1.1.4: A graph G is said to be *bipartite* if there exists a partition $V(G) = L \sqcup R$ such that every edge has exactly one endpoint in L and R .

We now present a simple strategy that works when G is bipartite.

Definition 1.1.5: Let $G = (L, R, E)$ be a bipartite graph and M be a matching. The graph G_M is defined as the directed graph formed from G by orienting each edge from L to R if it does not belong to M and from L to R , otherwise.

Lemma 1.1.4: Let M be a matching in a bipartite graph G and F be the set of free vertices. The M -augmenting paths are in one-to-one correspondence with directed paths from $L \cap F$ to $R \cap F$ in G_M .

Proof: Let P be a directed path from $L \cap F$ to $R \cap F$ in G_M . First, its endpoints are free with respect to M . Since G_M is bipartite, its edges alternate from edges $L \rightarrow R$ and edges $R \rightarrow L$. That is, the edges alternate from edges in M and not in M . Thus, the undirected edge set corresponding to P is an M -augmenting path.

Next, let P be an augmenting path. Note that P has odd length, and hence, its endpoints are on opposite sides of the partition (L, R) . Since its endpoints are also free, it suffices to show that P corresponds to a directed path in G_M . Note that every interior vertex of P has exactly one edge in M and one edge not in M . The former edge corresponds to a directed edge $R \rightarrow L$ and the latter corresponds to a directed edge $L \rightarrow R$. Thus, P corresponds to a directed path in G_M from $L \cap F$ and $R \cap F$. $\square$

This gives us a natural algorithm for maximum-matching in a bipartite graph. We merely build G_M and find a path from $L \cap F$ to $R \cap F$ using BFS/DFS. Building G_M and performing BFS/DFS both take $O(m + n)$ time. We may also assume $m \geq n/2$; otherwise, the graph contains isolated edges which can be removed in a preprocessing step taking $\mathcal{O}(n)$ times. Thus, overall, we get a $\mathcal{O}(mn)$ algorithm for maximum-matching in bipartite graphs.

1.2. Hopcroft-Karp Algorithm

Choosing only one augmenting path at a time is quite wasteful. The Hopcroft-Karp algorithm improves upon this by attempting to find many disjoint augmenting paths in every iteration. We start with the following generalization of [Corollary 1.1.1.1](#) and [Lemma 1.1.3](#).

Definition 1.2.1: A set of subgraphs $\{G_i\}$ are (*externally*) *vertex-disjoint* if they share no vertices. A set of paths $\{P_i\}$ are said to be *internally vertex-disjoint* if they share no vertices other than their endpoints.

Lemma 1.2.1: If G is a graph and M is a matching G then the following are equivalent for all $k \in \mathbb{N}$

- there exists a matching M' with $|M| + k$ edges;
- G contains k vertex-disjoint M -augmenting paths;
- there exists a matching M' such that the edge set $M \oplus M'$ contains at least k vertex-disjoint M -augmenting paths.

Proof:

- $a. \Rightarrow c.$

Consider M' such that $|M'| = |M| + k$. Note that $M \oplus M' = (M \setminus M') \sqcup (M' \setminus M)$ with $|M' \setminus M| = |M \setminus M'| + k$. Every vertex $v \in M \oplus M'$ has degree $\in \{1, 2\}$ as every vertex has degree at most 1 in each of M , M' and degree 1 in at least one of M , M' . Thus, $M \oplus M'$ must be a union of vertex-disjoint paths and cycles. As M and M' are matchings, each vertex of degree 2 in $M \oplus M'$ is adjacent to exactly one edge from M and one edge from M' . Since a cycle can be alternating only if it has even length,

it has an equal number of edges from M and M' . Since $|M' \setminus M| = |M \setminus M'| + k$, there must be at least k -many M -alternating paths in $M \oplus M'$.

- $c. \Rightarrow b.$

This is immediate by noting that $M \oplus M' \subseteq E(G)$.

- $b. \Rightarrow a.$

It suffices to show that if P and P' are vertex disjoint M -augmenting paths then P' is also a $(M \oplus P)$ -augmenting path. As P and P' are vertex-disjoint, they are edge disjoint too. Since endpoints of P' aren't adjacent to edges in M or P , they will be free with respect to $M \oplus P$ too. Next, any edge of $M \cap P' \subseteq M \setminus P$ is still in $M \oplus P$. Furthermore, any edge of $P' \setminus M$ isn't contained in M or P and hence, isn't an edge in $M \oplus P$. Thus, P' is $(M \oplus P)$ -alternating. □

Again, we shall assume G is bipartite. As always, we start with some helpful definitions to frame our algorithm.

Definition 1.2.2: In the graph G_M , define $L_0, L_1, \dots, L_\infty$ to be the levels in a breadth-first traversal rooted at $L \cap F$. That is,

$$\begin{aligned} L_0 &= L \cap F, \\ L_{i+1} &= \text{all vertices } v \notin \bigcup_{k=0}^i L_k \text{ s.t. } G_M \\ &\quad \text{contains an edge from } L_i \text{ to } v, \\ L_\infty &= \text{all vertices } v \notin \bigcup_{k=1}^n L_k. \end{aligned}$$

Definition 1.2.3: An edge $e = (u, v)$ is an *advancing edge* if $u \in L_i$ and $v \in L_{i+1}$. Otherwise, we call it a *retreating edge*. A path composed entirely of advancing edges is called an *advancing path*.

By definition, if e is an edge going from L_i to L_j for $j > i$, then we must have $j = i + 1$. Furthermore, L_i consists of all vertices that are at a distance of i from L_0 . The following lemma then somewhat clarifies the importance of advancing edges.

Lemma 1.2.2: If $e = (u, v)$ is an advancing edge we must have $d(v) > d(u) \Rightarrow d(v) = d(u) + 1$.

That is, an s - t path P is a shortest s - t path if and only if it is an advancing path.

Definition 1.2.4: A *blocking set* of M -augmenting paths is a maximal (under set-inclusion) set of vertex-disjoint advancing M -augmenting paths.

Hopcroft-Karp Algorithm

- 1 $M = \emptyset$
 - 2 **repeat**
 - 3 $\{P_i\}_k \leftarrow \textbf{find}$ a blocking set of M -augmenting paths
 - 4 $M \leftarrow M \oplus P_1 \oplus \dots \oplus P_k$
 - 5 **until** there is no M -augmenting path
-

As before, we know that this algorithm terminates in at most $\frac{n}{2}$ iterations as each iteration increases the cardinality of M by at least 1. To obtain a superior bound, we need to also integrate an alternative measure of progress. Particularly, we shall show that the length of the shortest augmenting path is strictly increasing every iteration. Intuitively, this occurs because augmenting by a path reverses the orientation of all its edges and hence, any future s - t path taking these edges must be longer than the shortest s - t paths available prior.

Lemma 1.2.3: If M_t and M_{t+1} are the matchings in two consecutive iterations of the Hopcroft-Karp algorithm, the length of the shortest M_{t+1} -augmenting path is strictly greater than the length of the shortest M_t -augmenting path.

Proof: Let $L_0, L_1, \dots$ denote the BFS layers of G_{M_t} and suppose $d \in \mathbb{N}$ is minimal such that L_d intersects $R \cap F$. Furthermore, suppose $P = u_0 u_1 \dots u_j$ is a shortest M_{t+1} -augmenting path. Note that u_0, u_j must be free with respect to M_t as once a vertex is matched, it remains matched after augmentation too. Below $d(\cdot)$ denotes the distance in G_{M_t} from $L \cap F$. Edges (u_i, u_{i+1}) were either

- a. advancing edges in G_{M_t} whose orientation is reversed in $G_{M_{t+1}}$ because (u_{i+1}, u_i) belonged to a blocking set $\mathcal{P}$

$$d(u_i) = d(u_{i+1}) + 1$$

- b. advancing edges in both G_{M_t}

$$d(u_{i+1}) = d(u_i) + 1$$

- c. reversing edges in G_{M_t}

$$d(u_{i+1}) < d(u_i)$$

Thus, in all cases we have $d(u_{i+1}) \leq d(u_i) + 1$. Inductively, $d \leq d(u_j) \leq j$. Note that $d(u_j) = j$ if and only if P is an advancing path in G_M . This implies that P is edge disjoint from $M_t \oplus M_{t+1}$. Since P wasn't used as an augmenting path in iteration t , it must be the case that there exists $v \in V(M_t \oplus M_{t+1}) \cap V(P)$. Note that v cannot be the endpoints of P as the endpoints of P are M_t -free. So, v is an interior vertex of P . As $v \in M_t \oplus M_{t+1}$, it must be adjacent to an edge $e \in M_t \oplus M_{t+1}$. However, as P is M_{t+1} -augmenting, for every interior vertex $v' \in P$, the incident edge from M_{t+1} is contained in P . Thus, $e \in M_t \oplus M_{t+1} \cap P$, contradicting the edge-disjointness of P from $M_t \cap M_{t+1}$. $\square$

Note that as augmenting paths are of odd length, each iteration increases the length of the shortest M -augmenting path by at least two. Using these measures of progress independently, we get the same bound of $n/2$ on the number of iterations. Using these two measures in different phases of the algorithm yields a superior bound.

Proposition 1.2.1: For any $t \leq \frac{n}{2}$, the number of iterations of Hopcraft-Karp is bounded by $t + n/2t$.

Proof: After t iterations, the length of the shortest M -augmenting path is at least $2t$. Then let $k = |M^*| - |M^t|$. Note that M must have at least one augmenting path with vertices at most $\frac{n}{k}$. Hence, $2t \leq \frac{n}{k} \Rightarrow k \leq \frac{n}{2t}$. Thus, the runtime is bounded by $t + k \leq t + \frac{n}{2t}$. $\square$

Using the AM-GM inequality, we note that $\sqrt{2n} \leq t + \frac{n}{2t}$ with equality obtained when $t = \sqrt{\frac{n}{2}}$.

Corollary 1.2.1.1: Hopcraft-Karp terminates in at most $\mathcal{O}(\sqrt{n})$ iterations.

1.3. Min-Cost Bipartite Perfect Matching, LP Relaxation

Minimum-Cost Bipartite Perfect Matching: Given an undirected bipartite graph $G = (L, R, E)$ together with costs $c : E \rightarrow \mathbb{R}_{\geq 0}$ for each edge, find a perfect matching M that minimizes $c(M) = \sum_{e \in M} c(e)$.

Without loss of generality, we shall assume that a perfect matching exists as we have already discussed algorithms capable of detecting this.

Definition 1.3.1: Let M be a matching. Then $c : E \rightarrow \mathbb{R}_{\geq 0}$ induces a cost function $c_M : E \rightarrow \mathbb{R}$ defined by

$$c_M(e) = \mathbb{1}\{e \notin M\} \cdot c(e) - \mathbb{1}\{e \in M\} \cdot c(e).$$

This induced cost is natural for M -augmenting paths due to the following lemma.

Lemma 1.3.1: If P is an M -augmenting path then $c(M \oplus P) = c(M) + c_M(P)$.

This motivates a greedy algorithm for our problem. Note that bipartiteness is not required yet.

Greedy Algorithm for Min-Cost Perfect Matching

```

1  $M = \emptyset$ 
2 while  $M$  is not a perfect matching do
3    $P \leftarrow$  find an  $M$ -augmenting path minimizing  $c_M(\cdot)$ 
4    $M \leftarrow M \oplus P$ 
5 end while

```

Correctness follows from a proof by induction showing that at iteration t , we have the minimum-cost matching of cardinality t .

Lemma 1.3.2: If M_k is a minimum-cost matching of size k in G and P is an M_k -augmenting path of minimum incremental cost, then $M_k \oplus P$ is a minimum-cost matching of size $k + 1$.

Proof: Let M_{k+1} denote the minimum-cost matching of size $k + 1$. By definition, we must have $c(M_{k+1}) \leq c(M_k \oplus P)$.

Now, we prove the reverse inequality. $M_k \oplus M_{k+1}$ is a union of alternating components that must contain at least one M_k -alternating path Q as $|M_{k+1}| = |M_k| + 1$. Let R denote the union of the remaining alternating components. Note that R must have the same number of edges in M_k and in M_{k+1} since the path Q already accounts for $|M_{k+1}| = |M_k| + 1$. So, $M_k \oplus R$ is a k -matching and $M_{k+1} \oplus R$ is a $(k + 1)$ -matching. From this, it is immediate that $c_{M_k}(R), c_{M_{k+1}}(R) \geq 0$ via

$$\begin{aligned} c(M_k) &\leq c(M_k \oplus R) = c(M_k) + c_{M_k}(R), \\ c(M_{k+1}) &\leq c(M_{k+1} \oplus R) = c(M_{k+1}) + c_{M_{k+1}}(R). \end{aligned}$$

Furthermore, we have $c_{M_k}(R) + c_{M_{k+1}}(R) = 0$ which implies $c_{M_k}(R) = c_{M_{k+1}}(R) = 0$.

$$\begin{aligned} c_{M_k}(R) + c_{M_{k+1}}(R) &= c(R \setminus M_k) - c(R \cap M_k) + c(R \setminus M_{k+1}) - c(R \cap M_{k+1}) \\ &= c(R \cap M_{k+1}) - c(R \cap M_k) + c(R \cap M_k) - c(R \cap M_{k+1}) \\ &= 0. \end{aligned}$$

Then, as Q is an M_k -alternating path, we have $c_{M_k}(P) \leq c_{M_k}(Q)$. Using this, we obtain our result

$$\begin{aligned}
c(M_k \oplus P) &= c(M_k) + c_{M_k}(P) \\
&\leq c(M_k) + c_{M_k}(Q) \\
&= c(M_k) + c_{M_k}(Q) + c_{M_k}(R) \\
&= c(M_k) + c_{M_k}(M_k \oplus M_{k+1}) \\
&= c(M_k \oplus (M_k \oplus M_{k+1})) \\
&= c(M_{k+1}).
\end{aligned}$$

□

Implementing this algorithm requires us to find shortest c_M -weighted alternating paths. Assuming G is bipartite, we can construct G_M and reduce the problem to finding the minimum-cost path in G_M from $L \cap F$ to $R \cap F$. A directed edge (u, v) has cost $c(u, v)$ if $(u, v) \notin M$ and has cost $-c(v, u)$ if

1.4. *Mise en Place* : Ski-Rental Problem

1.5. Online Matching

1.6. Algebraic Algorithms